

Design-Weighted Latent Class Analysis of Survey Data: An Adaptable Analysis Script

Openness to government negotiations as a worked example

Kevin

2026-06-25

1 Purpose and how to use this script

This is a complete, adaptable analysis for finding latent classes (hidden respondent types) in categorical survey items collected under a complex sampling design, estimating the class structure so that it describes the **population** rather than the achieved sample, attaching design-based confidence intervals to every estimated probability, writing class membership back to the respondents, and profiling that membership on demographics.

The worked example identifies types of respondents by their **openness to their government negotiating with a neighboring country**, from a multistage sample of about 1,200 respondents with base weights. Everything that is specific to a survey lives in one configuration block (Section 3). To apply this to your own data, edit that block and nothing else.

The script makes four methodological commitments, each explained where it occurs:

1. **The design is respected during estimation**, not only afterward. Class prevalences and item-response probabilities are estimated by *pseudo-maximum likelihood* with the base weights, which is the analogue of declaring `svyset psu [pweight = w], strata(stratum)` in Stata. This is the established design-based approach to latent class analysis under complex sampling (Patterson, Dayton, and Graubard 2002, *Journal of the American Statistical Association* 97(459):721–728; Vermunt and Magidson 2007, *Sociological Methods & Research* 36(1):87–111).
2. **poLCA is used as an unweighted benchmark**, because it cannot use design weights, and the gap between it and the weighted fit measures how informative the design is for this typology. A built-in equivalence check confirms the weighted estimator reproduces poLCA exactly when all weights are set to one.
3. **Uncertainty is design-based**. Confidence intervals on every conditional and inclusion probability come from the stratified jackknife (JKn) using replicate weights, which respects the stratification and clustering of a multistage design.
4. **“Don’t know” is treated as missing**, not as a response category. The model is fit on respondents who answered every item, which also makes the poLCA benchmark an exact comparison; respondents dropped for a missing answer are then assigned a class from their answered items (Section 14), so no one is discarded and nothing is imputed. The configuration block notes where to change this if your items call for it.

The code uses the tidyverse, contains no `for` or `while` loops (iteration is done with `purrr` and matrix algebra), and is written to fail loudly and informatively rather than silently on edge cases.

2 Setup

```
library(tidyverse)      # data manipulation and plotting
library(matrixStats)    # numerically stable row-wise log-sum-exp and cumulative
  ↪ sums
library(survey)         # complex-design objects, replicate weights, svyglm
library(poLCA)          # unweighted benchmark latent class model
library(clue)           # solve_LSAP: optimal label alignment across fits
library(nnet)           # multinom: multinomial profiling (ships with base R)

# All reusable machinery lives in one annotated source file: the plotting and
# data-prep helpers, the weighted EM engine, the BCH three-step correction, and
# the worked-example simulator. Keep survey_lca_source.R in the same folder as
# this document; the render fails if it is not on the path. Sourcing it here,
  ↪ after
# the libraries, means every function is available to the analysis chunks below.
source("survey_lca_source.R")

`%|||%` <- rlang::`%|||%` # null-coalescing helper (also defined in the source
  ↪ file)
# Reproducibility is controlled by cfg$seed (set in the configuration block):
  ↪ every
# random step below (the EM random starts, the poLCA starts, and the simulation)
  ↪ seeds
# itself from it, so the classes and their labels are identical across runs.

# poLCA attaches MASS, whose select() and some matrixStats verbs (count) mask
# their tidyverse equivalents. We namespace-qualify dplyr::select(),
  ↪ dplyr::count(),
# and dplyr::distinct() throughout. This is a deliberate, defensive choice.
```

3 Configuration: the only block you edit to reuse this script

Everything design-specific is gathered here. When `cfg$simulated` is `TRUE`, the worked example below the block generates synthetic data that matches this configuration; when you set it to `FALSE`, point `cfg$data` at your own data frame and the simulation and the truth comparison are skipped automatically. *Why this block matters: it is the single point of contact between you and the analysis. Every section downstream reads from `cfg`, so a correct configuration here is what makes the rest of the script reusable without edits.*

```

cfg <- list(
  # --- columns in YOUR data -----
  items   = paste0("q", 1:8),    # the categorical indicator variables for the LCA
  aux     = c("age_grp", "education", "region"), # demographics for profiling
  strata  = "stratum",           # stratum identifier (design)
  psu     = "psu",               # primary sampling unit / cluster identifier
  ↪ (design)
  weight  = "w",                 # base weight (design)

  # --- analysis choices -----
  K_range = 2:6,                 # candidate numbers of latent classes to compare
  n_starts = 20L,                # random starts per fit (guards against local
  ↪ maxima)
  seed    = 2026L,               # master random seed: fix it and the classes (and
  ↪ their
                                   # labels) are identical on every run of the same
                                   ↪ data
  ref_class = 1L,                # baseline class for the multinomial profile; set
  ↪ to ANY
                                   # class in 1..K (every odds ratio is "Class k vs
                                   ↪ this one")
  na_codes = NULL,               # response code(s) to treat as MISSING, e.g. 9
  ↪ for "Don't know";
                                   # leave NULL if your data already uses NA for
                                   ↪ missing/DK

  # --- data -----
  simulated = TRUE,               # TRUE: build the worked example (Section 5) and
  ↪ run the
                                   # truth comparison (Section 13). FALSE: use YOUR
                                   ↪ data and
  # skip both. This is the only switch between demo
  ↪ and real use.
  data = NULL                     # your data frame, required when simulated =
  ↪ FALSE; left NULL
                                   # and ignored when simulated = TRUE (the
                                   ↪ simulator fills it)
)

```

A note on the **items coding requirement**, which the script checks: each indicator must be a small set of consecutive integer codes starting at 1 over its *substantive* responses (for example 1,2,3,4 for a four-point item), and items may differ in how many categories they have. “Don’t know” is **not** a response category in this version: it is treated as missing data, set to NA (directly, or by listing its code in `cfg$na_codes`). Respondents with any missing item are held out of the model fit (Section 6) but are still assigned a class afterwards from their answered items (Section 14), so no one is discarded and nothing is imputed.

```
# theme_lca() (the colorblind-safe figure theme) lives in survey_lca_source.R.
# Only the two-color weighted/unweighted palette, which is specific to this
# document's labels, is defined here.
wu_pal <- setNames(viridisLite::viridis(2, begin = 0.2, end = 0.75),
  c("Weighted (population)", "Unweighted (poLCA)"))
```

4 The latent class model

A latent class model supposes that each respondent belongs to one of K unobserved classes, and that **within a class the items are statistically independent** (the local-independence assumption). Writing π_k for the share of the population in class k , and ρ_{jck} for the probability that a member of class k gives response c to item j , the probability of observing respondent i 's full pattern of answers is a mixture over the classes:

$$P(\mathbf{y}_i) = \sum_{k=1}^K \pi_k \prod_{j=1}^J \rho_{j y_{ij} k}, \quad \text{with } \sum_{c=1}^{C_j} \rho_{jck} = 1 \text{ for every item } j \text{ and class } k.$$

The π_k are the **inclusion (class-size) probabilities** and the ρ_{jck} are the **conditional (item-response) probabilities**; together they are the *measurement model*. Each item is treated as an unordered (nominal) categorical variable, which is exactly what **poLCA** assumes, so the benchmark in Section 12 compares like with like. The items may have different numbers of categories; the model handles that without any special configuration. Because the items are independent within a class, a respondent who is missing an item still contributes through the items they did answer, which is what lets the model retain everyone for scoring without imputation.

Why this section matters: it is the contract the rest of the script honors. The local-independence assumption is what makes the classes interpretable as a measurement model rather than an arbitrary clustering, and it is the same assumption the indicator screen (Section 11) and the partial-response scoring (Section 14) rely on; if it fails badly, the diagnostics there are where it shows up.

4.1 Why ordinary poLCA is not enough

poLCA maximizes the *unweighted* likelihood, so it estimates the class structure of the **achieved sample**. Under a complex design with unequal selection probabilities, the sample over- or under-represents parts of the population, and the unweighted estimates inherit that distortion. To describe the **population**, each respondent's contribution to the likelihood must be weighted by their base weight w_i , giving the *design-weighted pseudo-log-likelihood*

$$\ell_w(\theta) = \sum_i w_i \log \left(\sum_{k=1}^K \pi_k \prod_j \rho_{j y_{ij} k} \right).$$

Maximizing ℓ_w yields *pseudo-maximum-likelihood* estimates of the finite-population parameters. This is the standard design-based device for latent class models under complex sampling (Patterson,

Dayton, and Graubard 2002, *Journal of the American Statistical Association* 97(459):721–728), and it is what we implement below, because `poLCA` has no argument for weights.

5 Worked-example data

Why this section matters: it makes the script self-contained and testable. Because the data come from known parameters, Sections 9, 12, and 13 can show how closely the weighted estimator recovers the truth, which is the only honest way to demonstrate that the design-based machinery does what it claims. It runs only when `cfg$simulated` is `TRUE`; with your own data the script skips straight to preparation.

The worked example is a multistage sample that matches the configuration: 9 strata, about 20 primary sampling units in total, roughly 1,200 respondents, base weights that vary across strata, and a design that is **informative** because the more heavily weighted strata are also the ones least open to negotiation. Eight items measure openness, deliberately on **different scales** (some four-point, some three-point, some binary) to exercise the variable-length machinery. A small fraction of responses, under 5 percent per item, are “Don’t know” and are set to `NA`, so the example also exercises the missing-data handling.

The generating code is `simulate_survey_data()` in `survey_lca_source.R`; it avoids loops, drawing classes and item responses through vectorized inverse-CDF samplers. The chunk below calls it when `cfg$simulated` is `TRUE`, and otherwise checks that you supplied your own data.

```
# Obtain the analysis data. When simulated, the source-file routine builds the
# worked example from cfg$seed and also returns the population class prevalences
# (pop_pi_true), which only the truth comparison in Section 13 consumes. When not
# simulated, the data must already be in cfg$data.
if (isTRUE(cfg$simulated)) {
  sim_out      <- simulate_survey_data(cfg)
  cfg$data     <- sim_out$data
  pop_pi_true  <- sim_out$pop_pi_true
  K_true      <- sim_out$K_true
}
if (is.null(cfg$data))
  stop("cfg$data is NULL. Set cfg$simulated <- TRUE to use the worked example, ",
       "or assign your own data frame to cfg$data and set cfg$simulated <-  
↪ FALSE.")
```

6 Preparing and validating the data

Before any modeling, the script confirms the indicators are well coded and the design columns are present, stopping with an explicit message if anything is wrong. This is the robustness layer: it converts silent downstream errors into clear, early ones. *Why this section matters: latent class estimates are only as trustworthy as the coding underneath them. A mislabeled category or an unflagged missing code does not error, it quietly biases every prevalence, so catching it here is what separates a reproducible result from a plausible-looking wrong one.*

“Don’t know” is treated as missing. Any code listed in `cfg$na_codes` is set to NA, and each item is then recoded to consecutive integers from 1 over whatever substantive categories it has, so items of different lengths are all handled the same way. The model is **fit on complete cases**, the respondents who answered every item, which keeps estimation clean and makes the **poLCA** benchmark in Section 12 an exact comparison. Respondents held out here because of a missing answer are not discarded: Section 14 assigns each of them a class from the fitted model using only the items they did answer. With a small fraction missing, this loses little and imputes nothing.

```
# Pull the working data and confirm required columns exist.
dat <- cfg$data
stopifnot(is.data.frame(dat))
required_cols <- c(cfg$items, cfg$aux, cfg$strata, cfg$psu, cfg$weight)
missing_cols <- setdiff(required_cols, names(dat))
if (length(missing_cols) > 0)
  stop("These configured columns are not in the data: ", paste(missing_cols,
    ↪ collapse = ", "))

# Map any configured missing/DK codes to NA, then recode each item to consecutive
# integers from 1 over its substantive responses (NA preserved). Coerce profiling
# covariates to factors so the multinomial and its predictions behave
↪ consistently.
# (to_na() and recode_consecutive() are defined in survey_lca_source.R.)
dat_all <- dat |>
  mutate(across(all_of(cfg$items), ~ recode_consecutive(to_na(as.integer(.x),
    ↪ cfg$na_codes))),
    across(all_of(cfg$aux), as.factor))

cats <- map_int(cfg$items, ~ max(dat_all[[.x]], na.rm = TRUE))
names(cats) <- cfg$items

# Complete-case subset for fitting, confidence intervals, and profiling.
↪ Respondents
# with any missing item are scored later (Section 14) from their answered items.
complete_items <- stats::complete.cases(dat_all[, cfg$items, drop = FALSE])
dat_prepared <- dat_all[complete_items, , drop = FALSE]

# Report the per-item category counts and missing footprint.
tibble(item = cfg$items, categories = cats,
  n_missing = map_int(cfg$items, ~ sum(is.na(dat_all[[.x]])))) |>
  knitr::kable(align = "lrr")
```

item	categories	n_missing
q1	4	45
q2	4	41
q3	4	51
q4	3	44

item	categories	n_missing
q5	3	36
q6	3	30
q7	2	39
q8	2	34

Of 1200 respondents, 918 answered every item and enter the fit; the remaining 282 are assigned a class in Section 14.

7 The weighted EM estimator

The script estimates the measurement model with a weighted Expectation-Maximization (EM) algorithm. EM alternates two steps until the weighted log-likelihood stops improving.

The **E-step** computes each respondent's posterior probability of belonging to each class, given the current parameters:

$$p_{ik} = \frac{\pi_k \prod_j \rho_{j y_{ij} k}}{\sum_m \pi_m \prod_j \rho_{j y_{ij} m}}.$$

The **M-step** updates the parameters as *weighted* proportions, which is the only place the base weights enter:

$$\hat{\pi}_k = \frac{\sum_i w_i p_{ik}}{\sum_i w_i}, \quad \hat{\rho}_{jck} = \frac{\sum_i w_i p_{ik} \mathbf{1}(y_{ij} = c)}{\sum_i w_i p_{ik}}.$$

The $\hat{\rho}_{jck}$ denominator runs over respondents who answered item j ; because the model is fit on complete cases (Section 6), that is every respondent, so the sum is the same $\sum_i w_i p_{ik}$ written above. The implementation already restricts the sum correctly, which is what lets the same code score partial response patterns later.

Why this section matters: this is the engine. The weighting in the M-step is the single mechanism that moves every estimate from describing the achieved sample to describing the population, so its correctness propagates to every prevalence, profile, and interval downstream. Because the objective being maximized is the weighted *pseudo*-log-likelihood rather than an ordinary likelihood, these are *pseudo*-maximum-likelihood estimates. They are consistent for the population parameters, but, as Section 10 explains, their standard errors cannot be read off the curvature of the objective in the usual way and must instead come from replication.

Two implementation choices make this robust and loop-free. First, the E-step is computed on the log scale and normalized with the **log-sum-exp** identity, which prevents numerical underflow when many small probabilities are multiplied together. Second, the iteration is expressed as a **purrr::reduce fold** over iteration indices rather than a **while** loop: each step takes the current parameter state and returns the updated one, and the fold stops once the weighted log-likelihood stops improving. The same E-step is reused, with the fitted parameters held fixed, to assign classes to respondents outside the fitting sample (Section 14); because items are independent within a

class, a missing item simply drops out of the per-respondent product, so a respondent is scored on whatever items they answered.

The estimator is implemented in `survey_lca_source.R`, where the code is annotated line by line: `em_run()` is the weighted EM fold, `make_inputs()` builds the integer item vectors and one-hot category matrices it consumes, `posterior_of()` is the standalone E-step reused for scoring, `align_to()` fixes label order across fits with the Hungarian algorithm, and `fit_lca()` runs many random starts and keeps the best weighted log-likelihood. Keeping the machinery there leaves this document focused on what the estimator means rather than on its bookkeeping.

8 Choosing the number of classes

The number of classes K is not known in advance and is chosen by comparing a sequence of models. Three quantities guide that choice, and because they are routinely misread, each is explained here in the terms you would use with an analyst.

BIC (Bayesian Information Criterion) scores a model by its fit, penalized for the number of parameters it uses:

$$\text{BIC} = -2\hat{\ell} + \text{df} \cdot \ln n, \quad \text{df} = (K - 1) + K \sum_j (C_j - 1).$$

The first term rewards fit (a higher log-likelihood $\hat{\ell}$ makes it smaller), and the second term charges a price for every additional parameter, a price that grows with the sample size n . **Lower BIC is better.** You read the column down the candidate values of K and look for the **minimum**; often the values fall steeply, reach a low point, and rise again, and that low point is the recommended number of classes. The interpretation is “the number of classes that explains the response patterns well without adding classes that only fit noise.” BIC is the primary tool for this decision in latent class analysis.

AIC (Akaike Information Criterion) has the same form with a smaller, fixed penalty (2 per parameter instead of $\ln n$):

$$\text{AIC} = -2\hat{\ell} + 2 \text{df}.$$

It is reported for completeness, but because its penalty is lighter it tends to **favor too many classes**, so when AIC and BIC disagree, prefer BIC for a parsimonious, interpretable solution.

Relative entropy answers a different question. BIC and AIC ask “how many classes?”; entropy asks “once we have them, how cleanly are respondents sorted into them?”

$$E = 1 - \frac{\sum_i \sum_k p_{ik} \ln p_{ik}}{n \ln K}.$$

It runs from 0 to 1. A value near 1 means most respondents have a posterior probability close to 1 for a single class, so the classification is sharp; a value near 0 means respondents are spread across classes with no clear home. A common rule of thumb treats values above about 0.8 as good separation. Entropy is **not** a measure of fit and must **not** be used to choose K on its own: a two-class model almost always has higher entropy than a five-class model simply because fewer classes are easier to separate. Read it alongside BIC as a description of how usable the resulting classification is, not as a selection criterion.

All enumeration here uses the **weighted** fit, so the chosen K describes the population, not the achieved sample. To keep the criteria honest, the weighted pseudo-log-likelihood is put on the sample-size scale before $\hat{\ell}$ enters BIC and AIC: the weights are normalized to sum to n , so the fit term and the $\ln n$ penalty are measured in the same units. Without this the fit term would be inflated by the average weight and the criteria would lean toward too many classes. The script reports all three quantities across the candidate range and selects the K that minimizes the weighted BIC. *Why this section matters:* K is the one modeling choice with no default, and every later result is conditional on it, so a well-calibrated, defensible selection here is what keeps the prevalences and profiles from being artifacts of an over- or under-fit class structure.

```
# df_k() and entropy_R2() are defined in survey_lca_source.R.
w_vec <- dat_prepared[[cfg$weight]]

# Information criteria for a WEIGHTED pseudo-likelihood need the log-likelihood
# and
# the sample-size penalty on the same scale. The base weights here do not sum to
# the sample size, so the weighted log-likelihood is rescaled to the n-scale
# before
# forming AIC/BIC: multiplying by n / sum(w) is exactly fitting with weights
# scaled
# to sum to n, because the pseudo-likelihood is linear in the weights. This
# leaves
# the point estimates untouched (they are scale-invariant) and only calibrates
# model
# selection, which otherwise leans toward too many classes.
scale_ic <- nrow(dat_prepared) / sum(w_vec)

enum <- map_dfr(cfg$K_range, function(K) {
  fit <- fit_lca(dat_prepared, w_vec, cats, cfg$items, K, cfg$n_starts)
  ll_n <- fit$ll * scale_ic # log-likelihood on the
  # sum-to-n scale
  tibble(K = K,
    logLik = ll_n,
    df = df_k(K, cats),
    AIC = -2 * ll_n + 2 * df_k(K, cats),
    BIC = -2 * ll_n + df_k(K, cats) * log(nrow(dat_prepared)),
    entropy = entropy_R2(fit$post, K),
    converged = fit$converged)
})

K_star <- enum |> slice_min(BIC, n = 1) |> pull(K) |> first()

enum |>
  mutate(across(c(logLik, AIC, BIC), ~ round(.x, 1)),
    entropy = round(entropy, 3)) |>
  knitr::kable(align = "r",
    caption = "Weighted fit statistics by number of classes. Selection
  # is by minimum BIC.")
```

Table 2: Weighted fit statistics by number of classes. Selection is by minimum BIC.

K	logLik	df	AIC	BIC	entropy	converged
2	-7873.0	35	15816.0	15984.8	0.597	TRUE
3	-7854.6	53	15815.2	16070.8	0.654	TRUE
4	-7833.6	71	15809.2	16151.6	0.638	TRUE
5	-7815.1	89	15808.2	16237.3	0.673	TRUE
6	-7798.6	107	15811.1	16327.1	0.706	TRUE

The selected solution has **2 classes**. The figure shows BIC and entropy across the candidate range; BIC is read for selection and entropy for separation.

```
enum |>
  dplyr::select(K, BIC, entropy) |>
  pivot_longer(-K, names_to = "criterion", values_to = "value") |>
  mutate(criterion = recode(criterion, BIC = "Weighted BIC", entropy = "Relative
    ↪ entropy")) |>
  ggplot(aes(K, value)) +
  geom_line(linewidth = 0.6, color = viridisLite::viridis(1, begin = 0.4)) +
  geom_point(size = 2.4, color = viridisLite::viridis(1, begin = 0.4)) +
  facet_wrap(~ criterion, scales = "free_y") +
  scale_x_continuous(breaks = cfg$K_range) +
  labs(x = "Number of latent classes (K)", y = NULL) +
  theme_lca()
```

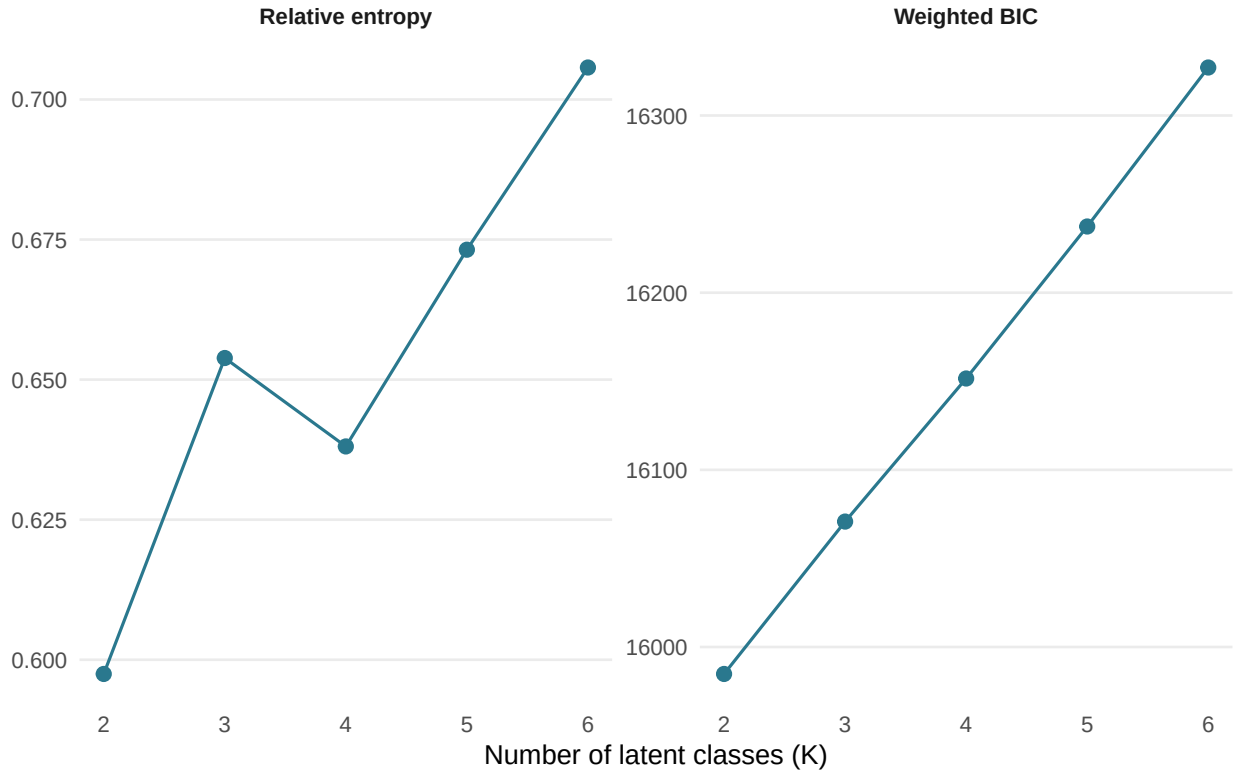


Figure 1: Weighted BIC and relative entropy across candidate numbers of classes. The chosen K minimizes BIC; entropy describes how cleanly respondents are classified.

9 The population measurement model

At the selected K , the script fits the model two ways on the same data: **weighted** (the population measurement model) and **unweighted** (which, fit with **poLCA**'s assumptions, is what **poLCA** itself targets). The comparison in Section 12 then shows what the design weights change. *Why this section matters: this is the deliverable. The prevalences and the response-probability profiles are what a reader actually quotes and names, and fixing the label order here is what keeps every subsequent table, figure, and interval referring to the same classes.*

The model has two sets of parameters, and both deserve a plain reading because they are the substantive output of the analysis.

The **inclusion probabilities** π_k are the **class prevalences**: the fraction of the population in each class. They sum to 1 across classes. A prevalence of 0.35 for a class means an estimated 35 percent of the population belongs to that type. These are the headline numbers, the size of each segment.

The **conditional probabilities** ρ_{jck} are **item-response probabilities**: given that a respondent is in class k , the probability they give answer c to item j . For each class and item these sum to 1 across the answer categories. They are how you *characterize and name* a class. If, in one class, the probability of the most-open answer is high on every negotiation item, that class is the “open” type; if another class concentrates on the guarded answers, that is the “closed” type. Reading down the conditional probabilities for a class is reading its profile.

Class labels are arbitrary. The algorithm could number the classes in any order, so “Class 1” carries no meaning on its own. The script does not impose a substantive order on the classes: with items of differing lengths there is no single scale on which to rank them. What it does do is fix one label order (from the weighted fit) and align the unweighted and **poLCA** fits to it (Section 12), so that “Class 2” refers to the same class in every table and figure. You read each class by its conditional-probability profile and name it yourself.

```
# Fit at K_star on the complete cases, weighted and unweighted. The weighted fit
↪ sets
# the label order; the unweighted fit is aligned to it so "Class k" is
↪ consistent.
fit_w <- fit_lca(dat_prepared, w_vec, cats, cfg$items, K_star, cfg$n_starts)
fit_u <- align_to(fit_lca(dat_prepared, rep(1, nrow(dat_prepared))), cats,
↪ cfg$items,
                                K_star, cfg$n_starts), fit_w)

# Posteriors recomputed from the FINAL parameters (the same kernel that scores
# out-of-sample respondents in Section 14), then written back so the same
↪ replicate
# design serves both the measurement-model CIs and the profiling regression.
inp_full <- make_inputs(dat_prepared, cfg$items, cats)
post_w <- posterior_of(fit_w$pi, fit_w$rho, inp_full$Y)
colnames(post_w) <- paste0("post_class", seq_len(K_star))
modal_int <- max.col(post_w, ties.method = "first")
dat_prepared <- dat_prepared |>
  bind_cols(as_tibble(post_w)) |>
  mutate(modal_class = factor(modal_int, levels = seq_len(K_star)),
         max_posterior = post_w[cbind(seq_len(n()), modal_int)])
```

10 Design-based confidence intervals via the stratified jackknife

Why this section matters: a prevalence without an interval invites false precision. The stratified jackknife is what turns the point estimates into defensible inference under the actual design, and it is the reason a typology built on few PSUs reports honestly wide intervals rather than misleadingly tight ones.

10.1 Why the survey weights alone are not enough

A natural question at this point is: the data already came with weights, so why not simply use them and be done? The answer is that survey weights do **one** of the two jobs that honest estimation requires, and the confidence intervals depend on the other.

The weights do the first job, **point estimation**: they make the prevalences and response probabilities describe the *population* rather than the achieved sample. That is precisely the weighted M-step in Section 7, $\hat{\pi}_k = \sum_i w_i p_{ik} / \sum_i w_i$, and the script does exactly that. If the only goal were the estimates themselves, “just use the weights” would be the complete answer.

The weights do **not** do the second job, **variance estimation**, because the precision of a survey estimate depends on *how the sample was selected*, not only on the weights. Two surveys can share an identical weight column yet differ greatly in precision: a sample of 1,200 people drawn as 20 clusters carries far less information than 1,200 people drawn independently, because respondents within a cluster tend to resemble one another, so each additional person in a cluster adds less new information than an independent person would. The weight attached to a respondent says nothing about which cluster they came from; the **design** (the stratum and PSU identifiers) is what carries that information. This is why the code below declares `svydesign(ids = ~psu, strata = ~stratum, weights = ~w)` and builds replicate weights from it, rather than working from the weight column alone.

The practical consequence of ignoring this is concrete and one-directional. If you weighted the data and then computed a standard error as though the weighted sample were a simple random sample of independent people, the estimate would be correct but the standard error would typically be **too small**, often by a factor of 1.5 to 2.5 in clustered samples. That inflation factor is the *design effect*. Using the weights for the estimate while ignoring the design for the variance, the single most common error in survey analysis, produces confidence intervals that are falsely narrow and overstates how much the data actually tell you.

There is a second, independent reason the variance cannot come from the weighted fit alone. The weighted objective $\sum_i w_i \log(\cdot)$ is a *pseudo*-log-likelihood, not a genuine likelihood for any probability model, because fractional and unequally weighted contributions do not correspond to a count of independent observations. The standard maximum-likelihood machinery that would ordinarily hand back standard errors from the curvature of the likelihood (the information matrix) therefore does not apply. The variance has to be estimated by *replication*, refitting the model on systematically perturbed versions of the design, which is what the stratified jackknife does and what the next subsection describes.

In one sentence for a non-technical audience: **the weights tell us who the population is, and the design tells us how sure we are**, and we need both.

10.2 Estimating the design-based variance

Confidence intervals must reflect the sampling design, not pretend the data are a simple random sample. The script uses **replicate weights**: the design is declared, converted to a set of stratified jackknife (JKn) replicates, and the entire weighted EM is re-estimated on each replicate. The spread of the replicate estimates gives the design-based variance,

$$\widehat{\text{Var}}(\hat{\theta}) = \sum_r c_r (\hat{\theta}^{(r)} - \hat{\theta})^2,$$

where each JKn replicate r deletes one primary sampling unit and reweights the rest of its stratum, with the scale factor c_r that **survey** supplies. With about 20 PSUs across 9 strata this produces roughly 20 replicates and about 11 degrees of freedom for the variance, which is the true precision of the design regardless of the 1,200 respondents; intervals will be honestly wide.

Two robustness points are built in. The design declaration uses `nest = TRUE` so PSUs nested within strata are handled correctly. And **lonely PSUs** (a stratum with a single PSU, which would break JKn) are handled by setting `options(survey.lonely.psu = "adjust")`, which centers such a

stratum's contribution at the grand mean rather than erroring; the script reports whether any were encountered. Each replicate refit is **warm-started at the full-sample solution and aligned to it**, which keeps the 20 refits stable, fast, and free of label switching.

```
options(survey.lonely.psu = "adjust")

# Report PSU structure so the analyst sees the degrees of freedom they actually
#   have.
# Note: dplyr::count() is namespace-qualified because matrixStats (loaded for
# rowLogSumExps) also exports a count(), and it loads after dplyr here.
psu_structure <- dat_prepared |>
  dplyr::distinct(.data[[cfg$strata]], .data[[cfg$psu]]) |>
  dplyr::count(.data[[cfg$strata]], name = "psus_in_stratum")
n_psu_total <- nrow(dplyr::distinct(dat_prepared, .data[[cfg$psu]]))
n_strata <- n_distinct(dat_prepared[[cfg$strata]])
lonely_any <- any(psu_structure$psus_in_stratum < 2)

# Declare the design and build stratified jackknife replicate weights.
des <- svydesign(ids = as.formula(paste0("~", cfg$psu)),
  strata = as.formula(paste0("~", cfg$strata)),
  weights = as.formula(paste0("~", cfg$weight)),
  data = dat_prepared, nest = TRUE)
rep_des <- as.svrepdesign(des, type = "JKn")

# Parameter bookkeeping as a TIBBLE, in the exact order flatten_fit() produces.
# This carries item/category/class as real columns, so nothing is recovered from
# string parsing later and the survey-question names are preserved downstream.
param_meta <- bind_rows(
  tibble(kind = "pi", item = NA_character_, category = NA_integer_, class =
    seq_len(K_star)),
  imap_dfr(cats, function(Cj, item_name)
    expand_grid(class = seq_len(K_star), category = seq_len(Cj)) |> # category
    varies fastest
    transmute(kind = "rho", item = item_name, category, class))
)
param_names <- param_meta |>
  mutate(nm = if_else(kind == "pi",
    sprintf("pi[%d]", class),
    sprintf("rho[%s,cat%d,class%d]", item, category, class)))
  pull(nm)
flatten_fit <- function(fit) set_names(c(fit$pi, unlist(map(fit$rho,
  as.vector)))), param_names)

# The statistic survey::withReplicates evaluates on each replicate: refit the
#   weighted EM
# on the replicate weights, warm-started at the full-sample fit and aligned to
#   it.
```

```

theta_fun <- function(w_rep, data) {
  inp <- make_inputs(data, cfg$items, cats)
  fit <- em_run(inp$Y, inp$OH, cats, w_rep, K_star,
               init = fit_w[c("pi", "rho")], maxit = 400L)
  flatten_fit(aligned_to(fit, fit_w))
}

rep_var <- withReplicates(rep_des, theta_fun)
est_vec <- flatten_fit(fit_w)
se_vec <- sqrt(diag(vcov(rep_var)))

# Tidy table of every probability with its design-based 95% CI:
  ↳ item/category/class
# are columns, ready to filter, join, and plot.
ci_tbl <- param_meta |>
  mutate(estimate = as.numeric(est_vec), se = as.numeric(se_vec),
         lo = pmax(0, estimate - 1.96 * se),
         hi = pmin(1, estimate + 1.96 * se))

```

The class-size (inclusion) probabilities, with their design-based intervals:

```

ci_tbl |>
  filter(kind == "pi") |>
  transmute(Class = paste("Class", class),
            Prevalence = round(estimate, 3),
            SE = round(se, 4),
            `95% CI` = sprintf("[% .3f, % .3f]", lo, hi)) |>
  knitr::kable(align = "lrrr",
               caption = "Population class prevalences (inclusion probabilities)
  ↳ with design-based JKn confidence intervals. Class labels are
  ↳ arbitrary; read each class by its profile (next figure).")

```

Table 3: Population class prevalences (inclusion probabilities) with design-based JK_n confidence intervals. Class labels are arbitrary; read each class by its profile (next figure).

Class	Prevalence	SE	95% CI
Class 1	0.474	0.0558	[0.365, 0.584]
Class 2	0.526	0.0558	[0.416, 0.635]

The item-response (conditional) probabilities are shown as a heatmap, which is far easier to scan than overlaid lines when there are several items: each panel is a class, each row an item, each column a response category, and the shade (and printed value) is the probability of that answer within that class. Reading a panel top to bottom gives that class's profile; comparing panels shows how the classes differ. Items with fewer categories simply leave the higher columns blank. The design-based confidence interval for every cell is in the exported table `lca_measurement_model_with_CIs.csv`; intervals are omitted from the heatmap only to keep it legible.

```

rho_plot <- ci_tbl |>
  filter(kind == "rho") |>
  mutate(Class = factor(paste("Class", class), levels = paste("Class",
    ↪   seq_len(K_star))),
    item = factor(item, levels = cfg$items),
    category = factor(category),
    lbl = sprintf("%.2f", estimate),
    lbl_col = if_else(estimate > 0.55, "white", "grey15"))

ggplot(rho_plot, aes(category, fct_rev(item), fill = estimate)) +
  geom_tile(color = "white", linewidth = 0.5) +
  geom_text(aes(label = lbl, color = lbl_col), size = 2.4) +
  scale_color_identity() +
  facet_wrap(~ Class, nrow = 1) +
  scale_fill_viridis_c(limits = c(0, 1), name = "P(response | class)") +
  labs(x = "Response category", y = NULL) +
  theme_lca() +
  theme(panel.grid = element_blank(),
    legend.key.width = unit(1.4, "cm"))

```

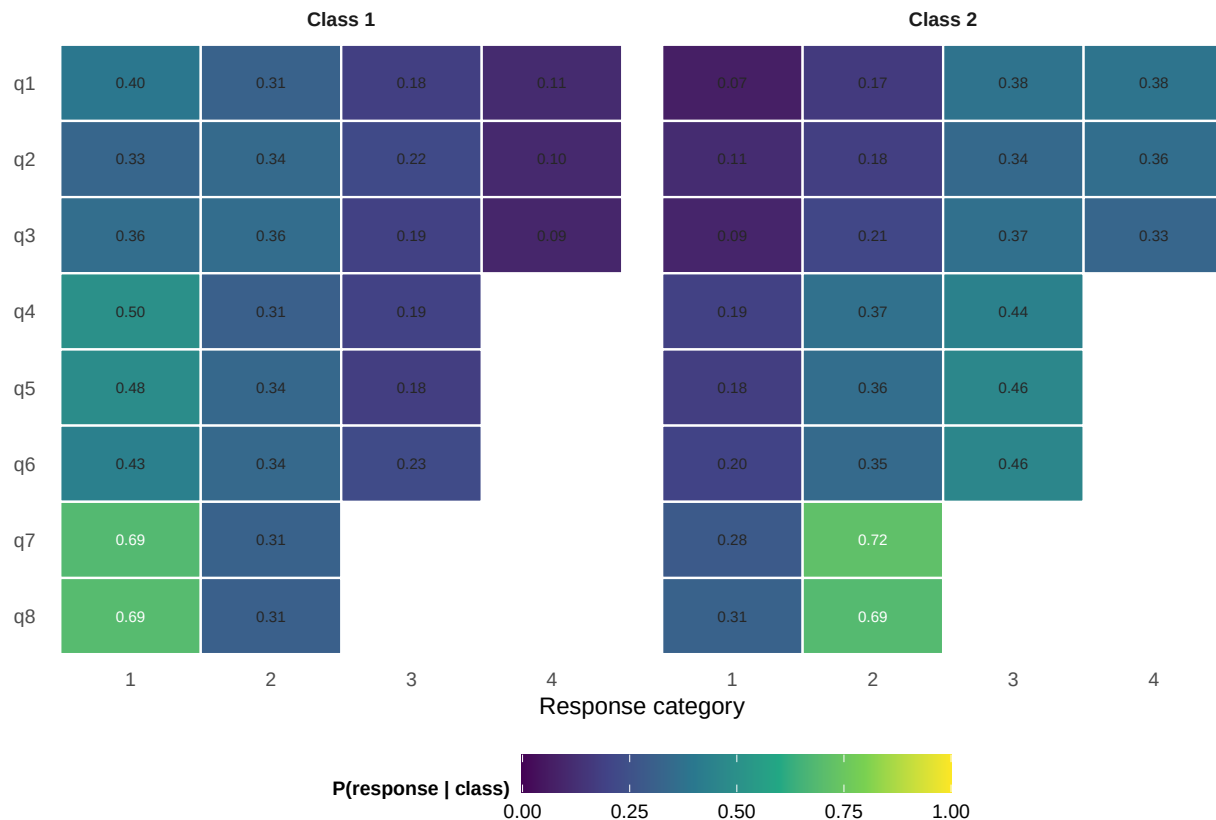


Figure 2: Population item-response (conditional) probabilities by class. Each panel is a class; rows are items, columns are response categories. Items of different lengths leave the higher columns blank. Cell shade and label give $P(\text{response} | \text{class})$. Class labels are arbitrary.

11 Which indicators carry the classification

Not every item earns its place in the model. An item helps define the classes only to the extent that the classes answer it *differently*; an item that every class answers the same way adds parameters and noise without sharpening the typology, and is a candidate to drop. The diagnostic below reads this directly off the fitted measurement model (the same ρ shown in the heatmap above).

For each item it computes a **discrimination** score: the average total-variation distance between the class-conditional response distributions, taken over all pairs of classes. For two classes a and b the total-variation distance on item j is $\frac{1}{2} \sum_c |\rho_{jca} - \rho_{jcb}|$, which is 0 when the two classes answer identically and 1 when their answers never overlap. Averaging over class pairs gives one number per item in $[0, 1]$: near 0 means the item does not separate the classes, larger means it does. The low items are the ones to consider removing; after dropping them you refit and confirm the class structure and its interpretation are essentially unchanged.

The discrimination score is an effect size; on its own it does not say how small is “too small.” The automated screen below turns it into a keep-or-drop decision with BIC. For each item it asks whether letting that item’s response probabilities differ across classes is worth the extra parameters, by comparing the fitted model against a restricted model in which the item is made *class-independent*.

(its response distribution shared across classes, so it cannot inform the classes). When BIC prefers the restricted model, the item is flagged as a weak contributor. This is the model-comparison logic of Dean and Raftery (2010, *Annals of the Institute of Statistical Mathematics* 62(1):11–35) in a simplified form. Their method embeds the same full-versus-restricted BIC comparison inside a stepwise (headlong) search over which variables to keep; the screen here runs it once per item against the full model rather than as a search. The `LCAvarese` package implements the further refinement of Fop, Smart, and Murphy (2017, *Annals of Applied Statistics* 11(4):2080–2110), which lets an excluded item depend on the other items instead of collapsing to its marginal as the simple restricted model here does. The size of the BIC gap grades the strength of the evidence: by Kass and Raftery (1995, *Journal of the American Statistical Association* 90(430):773–795), a difference beyond about 6 is strong and beyond about 10 very strong.

```
# Discrimination of each item under the fitted weighted model: how differently
  ↪ the classes
# answer it, measured as the mean over class pairs of the total-variation (TV)
  ↪ distance
# between their class-conditional response distributions. 0 = all classes answer
  ↪ alike (a
# drop candidate); near 1 = strong separation. Loop-free: map over items, then
  ↪ over pairs.
# Computed here; shown in the figure below and in the screen table further down.
class_pairs <- utils::combn(K_star, 2, simplify = FALSE) # list of c(a, b)
  ↪ class-index pairs

item_discrim <- tibble(
  item = cfg$items,
  discrimination = map_dbl(fit_w$rho, function(rho_j) {
    # rho_j is (categories x classes); rho_j[, k] is class k's response
    ↪ distribution.
    # TV distance for one pair of classes, then averaged over all pairs.
    mean(map_dbl(class_pairs, ~ 0.5 * sum(abs(rho_j[, .x[1]] - rho_j[, .x[2]]))))
  })
) |>
  arrange(desc(discrimination))
```

```
item_discrim |>
  mutate(item = factor(item, levels = rev(item))) |>
  ggplot(aes(discrimination, item)) +
  geom_col(fill = viridisLite::viridis(1, begin = 0.45), width = 0.7) +
  geom_vline(xintercept = 0.1, linetype = "dashed", color = "grey55") +
  scale_x_continuous(limits = c(0, 1)) +
  labs(x = "Discrimination (mean pairwise total-variation distance)", y = NULL) +
  theme_lca()
```

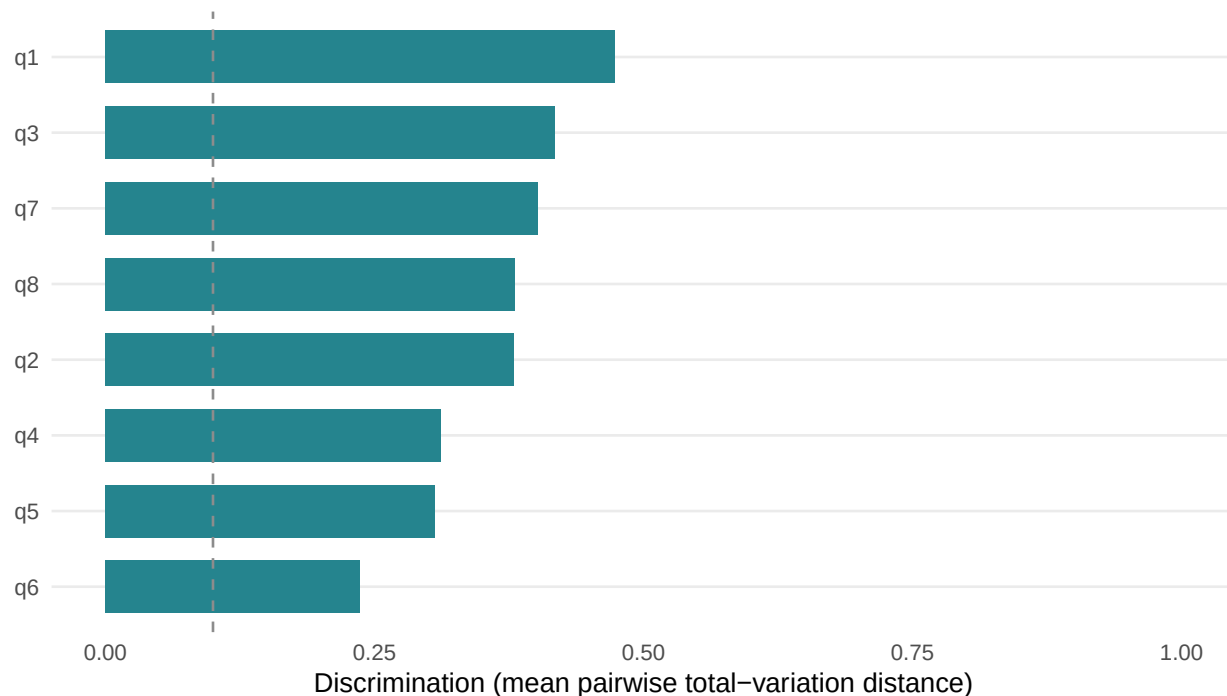


Figure 3: Item discrimination: how differently the latent classes answer each item (mean pairwise total-variation distance of the class-conditional response distributions). Bars near zero mark items that barely separate the classes and are candidates to drop before refitting. The dashed line at 0.1 is a soft reference, not a hard cutoff.

```
# Automated screen: does each item earn its class-DEPENDENT parameters by BIC?
  ↳ Compare, on
# the SAME respondents, the fitted model against a restricted model in which item
  ↳ j is made
# class-INDEPENDENT (its response distribution shared across classes, so it tells
  ↳ us nothing
# about the latent classes). Under local independence that restricted model
  ↳ factorises into
# an LCA on the OTHER items times item j's marginal, so its weighted
  ↳ pseudo-log-likelihood
# is ll(LCA without item j) + ll(item j's weighted marginal), and its BIC is
  ↳ directly
# comparable to the full model's. delta_bic = BIC(restricted) - BIC(full):
  ↳ NEGATIVE means
# BIC prefers dropping item j's class-dependence, so the item is flagged.
#
# This is the slowest chunk: it refits the LCA once per item. Lower cfg$n_starts
  ↳ here if you
# need speed, at some risk of a local optimum. It is reproducible because
  ↳ fit_lca() seeds
# each start from cfg$seed.
n_eff   <- nrow(dat_prepared)
```

```

# Put every log-likelihood on the same sum-to-n scale used for the criteria in
# Section 8 (scale_ic = n / sum(w_vec)), so the screen's BIC comparison is
# calibrated the same way as the K-selection BIC.
ll_full <- fit_w$ll * scale_ic
df_full <- df_k(K_star, cats) # df_k() was defined in Section
↪ 8

# Weighted log-likelihood of one item under a single (class-independent)
↪ distribution.
marginal_ll <- function(item) {
  y <- as.integer(dat_prepared[[item]])
  p <- as.numeric(xtabs(w_vec ~ y)); p <- p / sum(p) # weighted response
↪ proportions, 1..C
  sum(w_vec * log(p[y]))
}

bic_screen <- tibble(item = cfg$items) |>
  mutate(delta_bic = map_dbl(cfg$items, function(j) {
    keep <- setdiff(cfg$items, j)
    fit_wo <- fit_lca(dat_prepared, w_vec, cats[keep], keep, K_star,
↪ cfg$n_starts)
    ll_restr <- (fit_wo$ll + marginal_ll(j)) * scale_ic # sum-to-n
↪ scale, as above
    df_restr <- df_k(K_star, cats[keep]) + (cats[[j]] - 1) # item j: one
↪ shared distribution
    (-2 * ll_restr + df_restr * log(n_eff)) - (-2 * ll_full + df_full *
↪ log(n_eff))
  )))

# Combine the effect size and the decision into one ranked, flagged table.
item_screen <- item_discrim |>
  left_join(bic_screen, by = "item") |>
  mutate(flag = if_else(delta_bic < 0, "drop candidate", "keep")) |>
  arrange(delta_bic)

item_screen |>
  mutate(discrimination = round(discrimination, 3), delta_bic = round(delta_bic,
↪ 1)) |>
  knitr::kable(align = "lrrl",
    col.names = c("Item", "Discrimination", "Delta BIC", "Screen"),
    caption = "Automated indicator screen, most-droppable first.
↪ Discrimination is the mean pairwise total-variation distance
↪ (effect size). Delta BIC is BIC(item made class-independent)
↪ minus BIC(full model); a negative value means BIC prefers
↪ dropping the item's class-dependence, so it is flagged. The
↪ more negative the gap, the stronger the case (beyond about 6
↪ is strong, beyond 10 very strong).")

```

Table 4: Automated indicator screen, most-droppable first. Discrimination is the mean pairwise total-variation distance (effect size). Delta BIC is BIC(item made class-independent) minus BIC(full model); a negative value means BIC prefers dropping the item’s class-dependence, so it is flagged. The more negative the gap, the stronger the case (beyond about 6 is strong, beyond 10 very strong).

Item	Discrimination	Delta BIC	Screen
q6	0.236	32.9	keep
q4	0.312	56.2	keep
q5	0.306	62.0	keep
q8	0.381	72.5	keep
q2	0.379	74.5	keep
q7	0.402	84.1	keep
q3	0.417	94.7	keep
q1	0.474	114.4	keep

Treat a flag as a prompt, not a verdict. An item can be statistically weak yet worth keeping for face validity, and a binary item tends to score lower simply because it has less room to separate classes. Drop the items that are both flagged and substantively dispensable, set `cfg$items` to the survivors, refit, and confirm the prevalences and class profiles are essentially unchanged; if they move a lot, the item was carrying real information and belongs in the model.

A different item-set question is whether two items are *locally dependent*, more associated with each other than the shared classes explain. That is a model-fit diagnostic, the **bivariate residual**, not a contribution screen: it flags redundant or dependent *pairs* (where you might drop one of the pair, or model the dependence directly), not weak single items. It complements the screen above rather than replacing it, and a design-correct version needs a replicate-based reference distribution rather than the usual chi-square cutoff.

12 Benchmark against poLCA, and a validation check

Two comparisons with **poLCA** close the loop. First, a **validation**: when all weights are set to one, the weighted EM and **poLCA** maximize the identical likelihood, so their estimates must agree up to label order and convergence tolerance. Confirming this is the strongest evidence that the custom EM is implemented correctly. Second, a **substantive benchmark**: **poLCA** fit normally (unweighted) estimates the achieved-sample structure, and its gap from the weighted estimates measures how much the design matters for this typology. The script also reports the K that **poLCA**’s own BIC would select, since informative sampling can change the apparent number of classes. *Why this section matters: it does double duty. The unit-weight check is the script’s correctness test against an independent, trusted implementation, and the weighted-versus-poLCA gap is the headline quantity on real data, the visible imprint of the design on the class shares, which is what you keep when no ground truth exists.*

```
# poLCA needs a data frame of items coded 1..C with no NA (it uses na.rm).
polca_df <- dat_prepared |> dplyr::select(all_of(cfg$items)) |> as.data.frame()
```

```

polca_f <- as.formula(paste0("cbind(", paste(cfg$items, collapse = ", "), ") ~
  ↪ 1"))

# --- substantive benchmark: poLCA at K_star, unweighted ---
set.seed(cfg$seed + 7L)
pl <- poLCA(polca_f, data = polca_df, nclass = K_star, nrep = 10,
  maxiter = 5000, na.rm = TRUE, verbose = FALSE)
pl_aligned <- align_to(list(pi = as.numeric(pl$P),
  rho = map(pl$probs, t),
  post = pl$posterior), fit_w)

# --- poLCA's own BIC-selected K across the candidate range ---
polca_bic <- map_dfr(cfg$K_range, function(K) {
  set.seed(cfg$seed + 10L + K)
  m <- poLCA(polca_f, data = polca_df, nclass = K, nrep = 5,
    maxiter = 5000, na.rm = TRUE, verbose = FALSE)
  tibble(K = K, poLCA_BIC = m$bic)
})
polca_K <- polca_bic |> slice_min(poLCA_BIC, n = 1) |> pull(K) |> first()

# --- validation: weighted EM with all weights = 1 should match poLCA ---
fit_unit <- align_to(fit_lca(dat_prepared, rep(1, nrow(dat_prepared)), cats,
  ↪ cfg$items,
  K_star, cfg$n_starts), fit_w)
max_pi_gap <- max(abs(fit_unit$pi - pl_aligned$pi))
max_rho_gap <- max(map2_dbl(fit_unit$rho, pl_aligned$rho, ~ max(abs(.x - .y))))

tibble(`Latent class` = paste("Class", seq_len(K_star)),
  `Population (weighted)` = round(fit_w$pi, 3),
  `Sample (poLCA)` = round(pl_aligned$pi, 3),
  `Difference (pp)` = round((fit_w$pi - pl_aligned$pi) * 100, 1)) |>
  knitr::kable(align = "lrrr",
    caption = "Class prevalence: weighted (population) versus
  ↪ unweighted poLCA (achieved sample). The difference is the
  ↪ imprint of the design.")

```

Table 5: Class prevalence: weighted (population) versus unweighted poLCA (achieved sample). The difference is the imprint of the design.

Latent class	Population (weighted)	Sample (poLCA)	Difference (pp)
Class 1	0.474	0.411	6.3
Class 2	0.526	0.589	-6.3

The validation check returns a maximum class-size discrepancy of 7×10^{-4} and a maximum response-probability discrepancy of 4×10^{-4} between the unit-weight EM and poLCA; values near zero confirm the EM is correct. The weighted BIC selected 2 classes, and poLCA's unweighted BIC selected 2.

```

bind_rows(
  tibble(Class = paste("Class", seq_len(K_star)), prevalence = fit_w$pi,
    Estimator = "Weighted (population)"),
  tibble(Class = paste("Class", seq_len(K_star)), prevalence = pl_aligned$pi,
    Estimator = "Unweighted (poLCA)")
) |>
  ggplot(aes(Class, prevalence, fill = Estimator)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  scale_fill_manual(values = wu_pal) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(x = NULL, y = "Class prevalence", fill = NULL) +
  theme_lca()

```

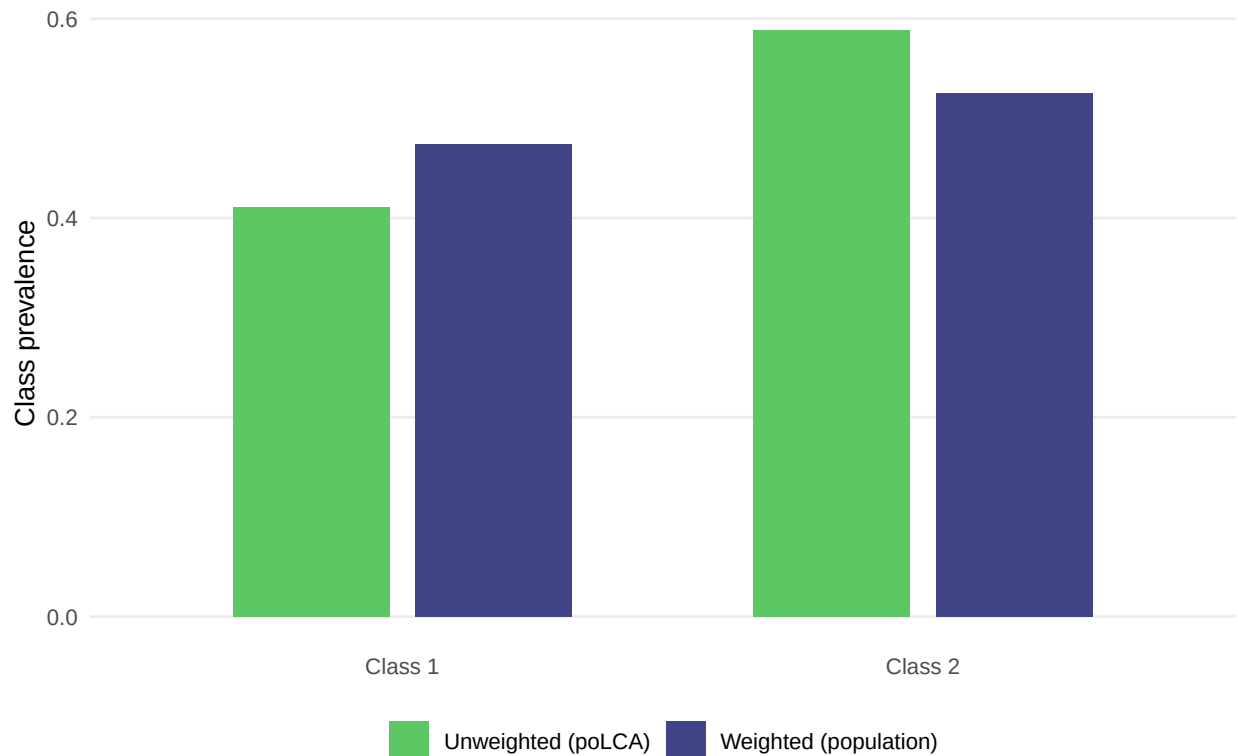


Figure 4: Population (weighted) versus achieved-sample (poLCA) class prevalences. Separation between the two indicates an informative design for this typology.

13 How much the design matters, against known truth

Because the worked example is simulated, the script can show the *gain* from weighting directly: how close each estimator's prevalences are to the population truth. On real data this section instead reports only the weighted-versus-unweighted divergence (Section 12), because the truth is unknown and no error can be computed without fabricating a target.

This comparison is only well defined when the selected number of classes equals the number used to generate the data. When the weighted BIC selects a different number (a central, weakly separated class can be merged under BIC, for instance), a class-by-class prevalence comparison has no meaning, and the script says so rather than forcing a misleading table. When the counts do match, the estimated classes are first aligned to the *true* generating classes by the weighted overlap of their modal assignments, so each row compares like with like.

```
# Runs only on the simulated worked example, where the generating classes and the
# population prevalences are known. With your own data (cfg$simulated = FALSE)
↪ the
# truth is unknown, so this prints nothing and Section 12 carries the comparison.
if (isTRUE(cfg$simulated)) {
  # Valid only when the selected K equals the generating K. Otherwise report that
  # plainly: a class-by-class comparison across different K is not defined.
  K_match <- (K_star == K_true)

  if (K_match) {
    # Align estimated classes to the TRUE classes by the weighted overlap of
    ↪ modal
    # assignment with the known generating class (Hungarian algorithm, maximizing
    ↪ overlap).
    ov_df <- tibble(wt = dat_prepared[[cfg$weight]],
                    est = max.col(post_w, ties.method = "first"),
                    tru = dat_prepared$true_class)
    overlap <- as.matrix(xtabs(wt ~ est + tru, data = ov_df))
    to_true <- as.integer(clue::solve_LSAP(overlap, maximum = TRUE)) # estimated
    ↪ row -> true col
    est_in_true_order <- integer(K_true)
    est_in_true_order[to_true] <- seq_len(K_star) # true col
    ↪ -> estimated row

    truth_tbl <- tibble(
      `Latent class (true)` = paste("Class", seq_len(K_true)),
      `Truth` = round(pop_pi_true, 3),
      `Weighted` = round(fit_w$pi[est_in_true_order], 3),
      `poLCA` = round(pl_aligned$pi[est_in_true_order], 3)) |>
      mutate(`|Weighted - Truth|` = abs(Weighted - Truth),
             `|poLCA - Truth|` = abs(poLCA - Truth))

    err_w <- round(max(truth_tbl$`|Weighted - Truth|`) * 100, 1)
    err_u <- round(max(truth_tbl$`|poLCA - Truth|`) * 100, 1)

    truth_text <- sprintf(
      "the weighted estimator's prevalences fall within %s percentage points of
      ↪ the population truth, against up to %s points for unweighted poLCA",
      err_w, err_u)

    display_tbl <- truth_tbl |>
```

```

    mutate(across(c(`|Weighted - Truth|`, `|poLCA - Truth|`), ~ round(.x, 3)))
  } else {
    err_w <- err_u <- NA_real_
    truth_text <- sprintf(
      "the weighted BIC selected %d classes while the data were generated with
      ↪ %d, so a class-by-class prevalence comparison is not defined here (this
      ↪ merging or splitting is itself an informative result)",
      K_star, K_true)
    display_tbl <- tibble(Note = truth_text)
  }

  knitr::kable(display_tbl, align = "l",
    caption = "Recovery of the known population prevalences
    ↪ (simulation only).")
} else {
  truth_text <- paste("this comparison runs only on the simulated worked
  ↪ example;",
    "with your own data the population truth is unknown, so
    ↪ the",
    "weighted-versus-unweighted divergence in Section 12 is the
    ↪ relevant measure")
}

```

Table 6: Recovery of the known population prevalences (simulation only).

Note

the weighted BIC selected 2 classes while the data were generated with 3, so a class-by-class prevalence comparison is not defined here (this merging or splitting is itself an informative result)

In this run, the weighted BIC selected 2 classes while the data were generated with 3, so a class-by-class prevalence comparison is not defined here (this merging or splitting is itself an informative result). *Why this section matters: it is the script's own proof that the weighting earns its place. When the truth is known, you can see the weighted estimator land closer to it than the unweighted one under an informative design; that is the empirical justification for everything the script does, and it is why the demonstration runs on simulated data where a target exists.* With your own data, set `cfg$simulated <- FALSE` and this section steps aside.

14 Assigning classes to respondents outside the fitting sample

The measurement model is a fixed set of parameters once it is estimated: the class sizes π_k and the item-response probabilities ρ_{jck} . Any respondent who answered the indicator items can be assigned a posterior probability of each class by applying the **same E-step** with those parameters held fixed,

$$p_{ik} = \frac{\pi_k \prod_j \rho_{j y_{ij} k}}{\sum_m \pi_m \prod_j \rho_{j y_{ij} m}},$$

and then taking the largest as their modal class. This is not a separate prediction model trained on demographics; it is the latent class model itself, used to *score* new response patterns, so it stays within the inferential framework. It applies to two groups: respondents held out of the fit here because they were missing an item, and entirely new respondents from a later wave or a separate file. Because the items are independent within a class, a missing answer drops out of the product, so a respondent is scored on whatever items they did answer; their maximum posterior will tend to be lower, reflecting that fewer items carry less information. *Why this section matters: it is what lets you fit on clean complete cases without throwing anyone away. Everyone receives a class from the same model, so the typology can be carried into later waves or merged files, and no one is dropped from the substantive story for a single missing answer.*

```
# Score EVERY respondent, including those held out of the fit for a missing item,
  ↪ from
# the fitted weighted model, using only their answered items.
dat_scored_all <- score_lca(dat_all, fit_w, cfg$items)
n_out <- sum(!complete_items)

# Preview a few held-out respondents with their assigned posteriors (NA = item
  ↪ not answered).
dat_scored_all[!complete_items, , drop = FALSE] |>
  dplyr::select(all_of(cfg$psu), all_of(cfg$items),
                starts_with("post_class"), modal_class, max_posterior) |>
  head(6) |>
  mutate(across(c(starts_with("post_class"), max_posterior), ~ round(.x, 3))) |>
  knitr::kable(align = "r",
                caption = "Class assignment for respondents held out of the fit
  ↪ (missing at least one item), scored from their answered
  ↪ items.")
```

Table 7: Class assignment for respondents held out of the fit (missing at least one item), scored from their answered items.

psu	q1	q2	q3	q4	q5	q6	q7	q8	post_class1	post_class2	modal_class	max_posterior
1	2	NA	1	2	3	3	NA	2	0.322	0.678	2	0.678
1	3	3	3	1	1	NA	1	1	0.847	0.153	1	0.847
1	3	4	2	NA	3	3	1	1	0.174	0.826	2	0.826
1	1	1	NA	2	2	2	1	2	0.923	0.077	1	0.923
1	3	4	NA	1	2	2	1	1	0.615	0.385	1	0.615
1	4	3	4	3	3	3	2	NA	0.002	0.998	2	0.998

Here 282 respondents who were not in the fit are assigned a class this way. To score a fresh data frame `newdat` that carries the same item columns, recoded the same way and with the same missing

handling, call `score_lca(newdat, fit_w, cfg$items)`; it returns the data with one posterior column per class, the modal class, and the maximum posterior.

15 Profiling class membership on demographics

The posterior class probabilities and the modal class were written back onto the data in Section 9, so each respondent now carries their most likely type and the full posterior. Per the analysis plan, **no posterior threshold is applied**: every respondent is kept, so the profiling sample is the full design sample and the weights retain their meaning. The task now is to describe *who* falls in each class, in terms of demographics.

15.1 Can a demographic used in the weighting also be a predictor here?

Yes. A variable such as age that was used to construct the survey weights can also appear as a predictor in this regression, and doing so is not double-counting. The weights and the covariate play different roles: the weights make the sample represent the population, and the covariate measures an association *within* that represented population. In fact, including the variables that drive the design as predictors is often the *better* choice. Pfeffermann (1993, *International Statistical Review* 61(2):317–337) and Gelman (2007, *Statistical Science* 22(2):153–164) make the underlying point: when the model conditions on the variables related to selection, the sampling becomes (approximately) ignorable, the weighted and unweighted estimates converge, and the analysis is less sensitive to the weights. So if age shaped the weights, putting age in the model is a feature, not a violation, and it generally improves efficiency.

Two qualifications. First, interpret the coefficient conditionally: it is the association of age with class membership *holding the other predictors fixed*, not a marginal effect. Second, the design-based standard errors (from the replicate weights below) remain the correct ones whether or not a predictor overlaps with the weighting variables. The circularity that genuinely must be avoided is a different one, raised earlier: a variable used as an **indicator** of the latent classes must not also be a **predictor** of them, because then you are explaining the classes with the very items that defined them. Demographics used in weighting are fine; the survey items that built the typology are not.

15.2 A design-based multinomial model for more than two classes

Because membership has more than two categories, the profile uses a **multinomial logistic regression**: it models the log-odds of each class against a reference class (here Class 1) as a function of the demographics. This generalizes the two-class logistic to the full set of classes in one model.

Getting design-correct uncertainty for a multinomial under a complex design is the same problem solved for the measurement model, and it is solved the same way. Within each jackknife replicate the coefficients are estimated by weighted multinomial likelihood (`nnet::multinom` with the replicate weights), and `survey::withReplicates` aggregates the spread of those replicate estimates into design-based standard errors, using the **same JK_n replicate weights** built in Section 10. This keeps the whole analysis on one coherent variance footing rather than mixing methods.

The same two cautions apply and are not hidden. The outcome is the *modal* class, an estimated quantity, so regressing it on covariates **attenuates** the coefficients toward zero relative to the true class-covariate association; this is a conservative, descriptive profile, not unbiased inference. For genuinely inferential statements, the bias-adjusted three-step methods (BCH and the maximum-likelihood correction) replace the modal regression while keeping this design-based variance, as the BCH subsection at the end of this section does. A simpler alternative, if a single segment is the whole question, is a binary design-based logistic regression (`svyglm` with `family = quasibinomial()`) of that one class against the rest.

```
# The baseline class for every odds ratio below is cfg$ref_class (set in the
  ↪ configuration
# block). It can be ANY class in 1..K_star: each comparison is then "Class k
  ↪ versus this
# class". Because the labels are reproducible once cfg$seed is fixed, the choice
  ↪ is stable
# from run to run. Guard against a value the selected number of classes cannot
  ↪ support,
# since K_star is chosen by BIC and may be smaller than you expect.
if (!(cfg$ref_class %in% seq_len(K_star)))
  stop("cfg$ref_class is ", cfg$ref_class, ", but only classes 1..", K_star,
    " exist (the number of classes is chosen by BIC). Set cfg$ref_class within
    ↪ 1..", K_star, ".")

# Put the reference level first so nnet::multinom uses it as the baseline.
ref_lvls <- c(cfg$ref_class, setdiff(seq_len(K_star), cfg$ref_class))
dat_prepared <- dat_prepared |>
  mutate(modal_class = factor(as.integer(as.character(modal_class)), levels =
    ↪ ref_lvls))

# Rebuild the design so its data carry the releveled outcome, then JKn
  ↪ replicates.
prof_des <- svydesign(ids = as.formula(paste0("~", cfg$psu)),
  strata = as.formula(paste0("~", cfg$strata)),
  weights = as.formula(paste0("~", cfg$weight)),
  data = dat_prepared, nest = TRUE)
prof_rep <- as.svrepdesign(prof_des, type = "JKn")
prof_formula <- reformulate(cfg$aux, response = "modal_class")

# Statistic that survey::withReplicates evaluates on each replicate. It refits
  ↪ the
# weighted multinomial and returns the coefficients as ONE named numeric vector,
  ↪ because
# withReplicates needs a vector (it then forms the covariance matrix across
  ↪ replicates).
# Names are "<class>:<term>", laid out one class at a time. To see the layout,
  ↪ run this
# on the full sample directly: multinom_theta(weights(prof_des), dat_prepared).
multinom_theta <- function(w_rep, data) {
```

```

data[[".w"]] <- as.numeric(w_rep) # inject the weights so multinom
↪ finds them in `data`
m <- nnet::multinom(prof_formula, data = data, weights = .w, trace = FALSE,
↪ maxit = 1000)
cf <- coef(m) # (K-1) x terms matrix; a plain
↪ named vector if K = 2
if (is.null(dim(cf)))
  cf <- matrix(cf, nrow = 1, dimnames = list(m$lev[2], names(cf)))
flat <- as.vector(t(cf)) # unroll class by class
names(flat) <- as.vector(t(outer(rownames(cf), colnames(cf), paste, sep =
↪ " : ")))
flat
}
mn_rep <- withReplicates(prof_rep, multinom_theta)
mn_est <- multinom_theta(weights(prof_des), dat_prepared) # named full-sample
↪ estimates
mn_se <- sqrt(diag(vcov(mn_rep))) # design SEs, same
↪ order
stopifnot(length(mn_est) == length(mn_se))

mn_tbl <- tibble(coef = names(mn_est),
                 estimate = as.numeric(mn_est),
                 se = as.numeric(mn_se)) |>
  separate(coef, into = c("class", "term"), sep = " : ") |>
  filter(term != "(Intercept)") |>
  mutate(comparison = paste0("Class ", class, " vs ", cfg$ref_class),
         OR = exp(estimate),
         lo = exp(estimate - 1.96 * se),
         hi = exp(estimate + 1.96 * se))

```

```

mn_tbl |>
  transmute(Comparison = comparison, Predictor = term,
            `Log-odds` = round(estimate, 3), SE = round(se, 3),
            `Odds ratio` = round(OR, 2),
            `OR 95% CI` = sprintf("%.2f, %.2f", lo, hi)) |>
  knitr::kable(align = "llrrrr",
               caption = paste0("Design-based multinomial profile of class
↪ membership on demographics, each class against Class ",
                                cfg$ref_class, ". Coefficients are descriptive
↪ and attenuated by classification error.))

```

Table 8: Design-based multinomial profile of class membership on demographics, each class against Class 1. Coefficients are descriptive and attenuated by classification error.

Comparison	Predictor	Log-odds	SE	Odds ratio	OR 95% CI
Class 2 vs 1	age_grp35-54	0.226	0.319	1.25	[0.67, 2.34]
Class 2 vs 1	age_grp55+	0.062	0.145	1.06	[0.80, 1.41]
Class 2 vs 1	educationHS or less	0.050	0.206	1.05	[0.70, 1.57]
Class 2 vs 1	educationSome college	0.162	0.162	1.18	[0.86, 1.61]
Class 2 vs 1	regionInterior	-0.514	0.148	0.60	[0.45, 0.80]

```
ggplot(mn_tbl, aes(OR, term, color = comparison)) +
  geom_vline(xintercept = 1, linetype = "dashed", color = "grey55") +
  geom_pointrange(aes(xmin = lo, xmax = hi), size = 0.35, linewidth = 0.4,
    position = position_dodge(width = 0.5)) +
  facet_wrap(~ comparison) +
  scale_x_log10() +
  scale_color_manual(values = setNames(viridisLite::viridis(max(1, K_star - 1),
    ↪ begin = 0.1, end = 0.75),
    unique(mn_tbl$comparison)), guide =
    ↪ "none") +
  labs(x = "Odds ratio versus reference class (log scale)", y = NULL) +
  theme_lca()
```

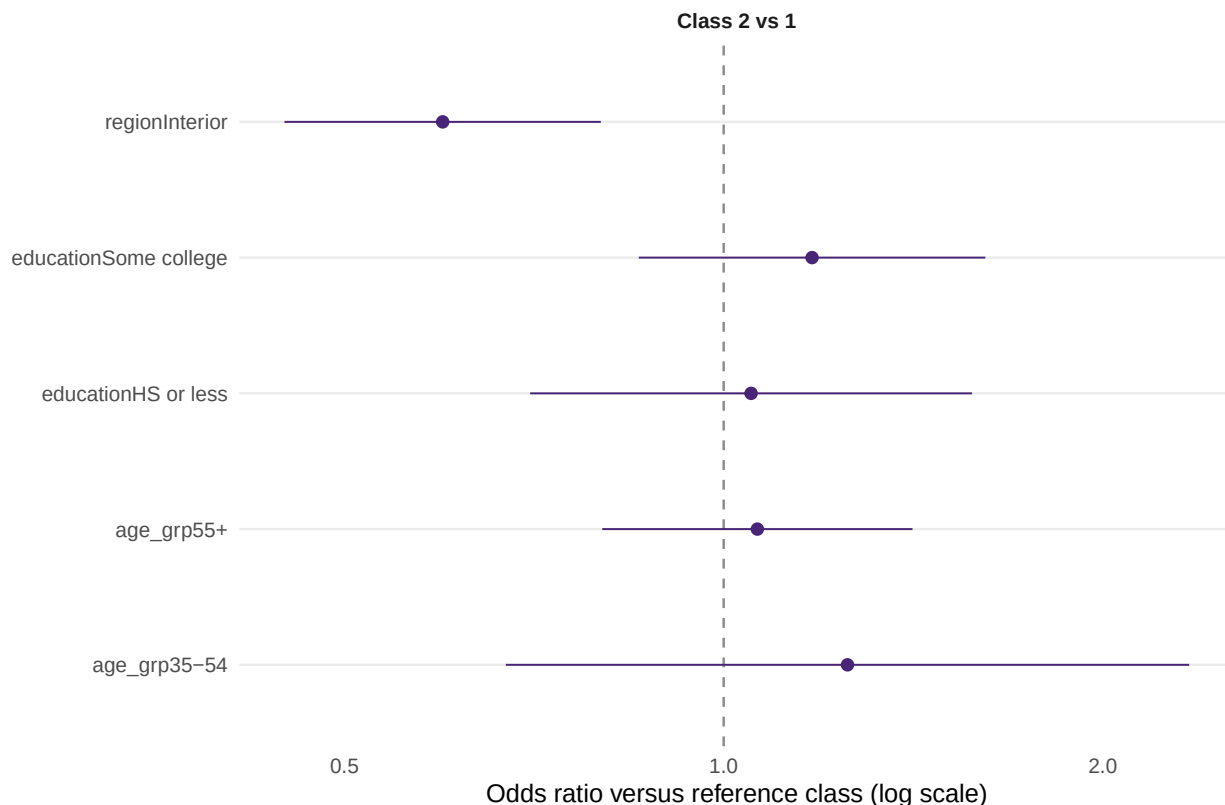


Figure 5: Design-based odds ratios with 95% intervals from the multinomial profile, each non-reference class versus the reference class, by predictor. Points right of 1 indicate higher odds of that class (versus reference) for that demographic level. Read as a descriptive, design-correct profile, attenuated by classification error.

15.3 Predicted probabilities of class membership by demographic

Odds ratios answer “how do the odds of this class versus the reference change with a predictor,” but they are awkward to read directly and say nothing about the reference class itself. The more legible summary, and the one asked for here, is the **predicted probability** of each class at given demographic values. The model already implies these: for any covariate profile the multinomial returns a full set of class probabilities that sum to one. The grid below varies each predictor across its levels while holding the others at their reference level, so each row isolates one demographic’s association with class membership. Predicted probabilities are recomputed on every jackknife replicate, so their confidence intervals are design-based, exactly like the odds ratios.

```
# Build the profile grid in two readable steps.
# Step 1: a one-row template with every predictor at its reference (first) level.
ref_row <- dat_prepared |>
  summarise(across(all_of(cfg$aux), ~ factor(levels(.x)[1], levels =
    ↪ levels(.x))))
# Step 2: for each predictor, repeat that template once per level and overwrite
↪ that one
```

```

# predictor with its levels. pvar/plevel record which predictor and level each
  ↪ row shows.
prof_grid <- map_dfr(cfg$aux, function(v) {
  lv <- levels(dat_prepared[[v]])
  ref_row |>
    slice(rep(1L, length(lv))) |>
    mutate(!v := factor(lv, levels = lv), pvar = v, plevel = lv)
}) |>
  mutate(prow = row_number())

# Statistic for withReplicates: refit the multinomial, then predict the class
# probabilities at every grid row. predict() returns a (grid rows x classes)
  ↪ matrix, or a
# plain vector when there are only two classes, which we widen back to two
  ↪ columns. We
# flatten it column by column into a named vector "<grid row>@@<class>" so
  ↪ withReplicates
# can attach one variance to each (grid row, class); pp_tbl splits the names back
  ↪ out.
prob_theta <- function(w_rep, data) {
  data[[".w"]] <- as.numeric(w_rep)
  m <- nnet::multinom(prof_formula, data = data, weights = .w, trace = FALSE,
  ↪ maxit = 1000)
  pp <- predict(m, newdata = prof_grid, type = "probs")
  if (is.null(dim(pp))) pp <- cbind(1 - pp, pp) # K = 2: vector P(2nd class)
  ↪ -> two columns
  colnames(pp) <- m$lev
  flat <- as.vector(pp)
  names(flat) <- as.vector(outer(seq_len(nrow(pp)), colnames(pp), paste, sep =
  ↪ "@@"))
  flat
}
pp_rep <- withReplicates(prof_rep, prob_theta) # design-based variance
  ↪ of every predicted prob
pp_est <- prob_theta(weights(prof_des), dat_prepared) # point estimates
  ↪ (full-sample fit)
pp_se <- sqrt(diag(vcov(pp_rep)))

# Split the names back into grid row + class, attach the CI, and label each row.
pp_tbl <- tibble(key = names(pp_est), p = as.numeric(pp_est), se =
  ↪ as.numeric(pp_se)) |>
  separate(key, into = c("prow", "class"), sep = "@@", convert = TRUE) |>
  mutate(lo = pmax(0, p - 1.96 * se), hi = pmin(1, p + 1.96 * se)) |>
  left_join(dplyr::select(prof_grid, prow, pvar, plevel), by = "prow")

pp_tbl |>
  transmute(Variable = pvar, Level = plevel, Class = paste("Class", class),

```

```

`P(class)` = round(p, 3),
`95% CI` = sprintf("[%0.3f, %0.3f]", lo, hi)) |>
arrange(Variable, Level, Class) |>
knitr::kable(align = "lllrr",
  caption = "Design-based predicted probability of membership in
  ↪ each class by demographic level, holding the other predictors
  ↪ at their reference level. Intervals are from the JKn replicate
  ↪ weights.")

```

Table 9: Design-based predicted probability of membership in each class by demographic level, holding the other predictors at their reference level. Intervals are from the JKn replicate weights.

Variable	Level	Class	P(class)	95% CI
age_grp	18-34	Class 1	0.439	[0.314, 0.564]
age_grp	18-34	Class 2	0.561	[0.436, 0.686]
age_grp	35-54	Class 1	0.384	[0.294, 0.475]
age_grp	35-54	Class 2	0.616	[0.525, 0.706]
age_grp	55+	Class 1	0.424	[0.316, 0.532]
age_grp	55+	Class 2	0.576	[0.468, 0.684]
education	BA+	Class 1	0.439	[0.314, 0.564]
education	BA+	Class 2	0.561	[0.436, 0.686]
education	HS or less	Class 1	0.427	[0.368, 0.486]
education	HS or less	Class 2	0.573	[0.514, 0.632]
education	Some college	Class 1	0.400	[0.264, 0.535]
education	Some college	Class 2	0.600	[0.465, 0.736]
region	Coastal	Class 1	0.439	[0.314, 0.564]
region	Coastal	Class 2	0.561	[0.436, 0.686]
region	Interior	Class 1	0.567	[0.440, 0.694]
region	Interior	Class 2	0.433	[0.306, 0.560]

```

pp_tbl |>
  mutate(Class = factor(paste("Class", class), levels = paste("Class",
    ↪ seq_len(K_star)))) |>
  ggplot(aes(plevel, p, color = Class)) +
  geom_pointrange(aes(ymin = lo, ymax = hi), size = 0.35, linewidth = 0.4,
    position = position_dodge(width = 0.5)) +
  facet_wrap(~ pvar, scales = "free_x") +
  scale_color_viridis_d(begin = 0.1, end = 0.8) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(x = NULL, y = "Predicted probability of class membership", color = NULL) +
  theme_lca() +
  theme(axis.text.x = element_text(angle = 20, hjust = 1))

```

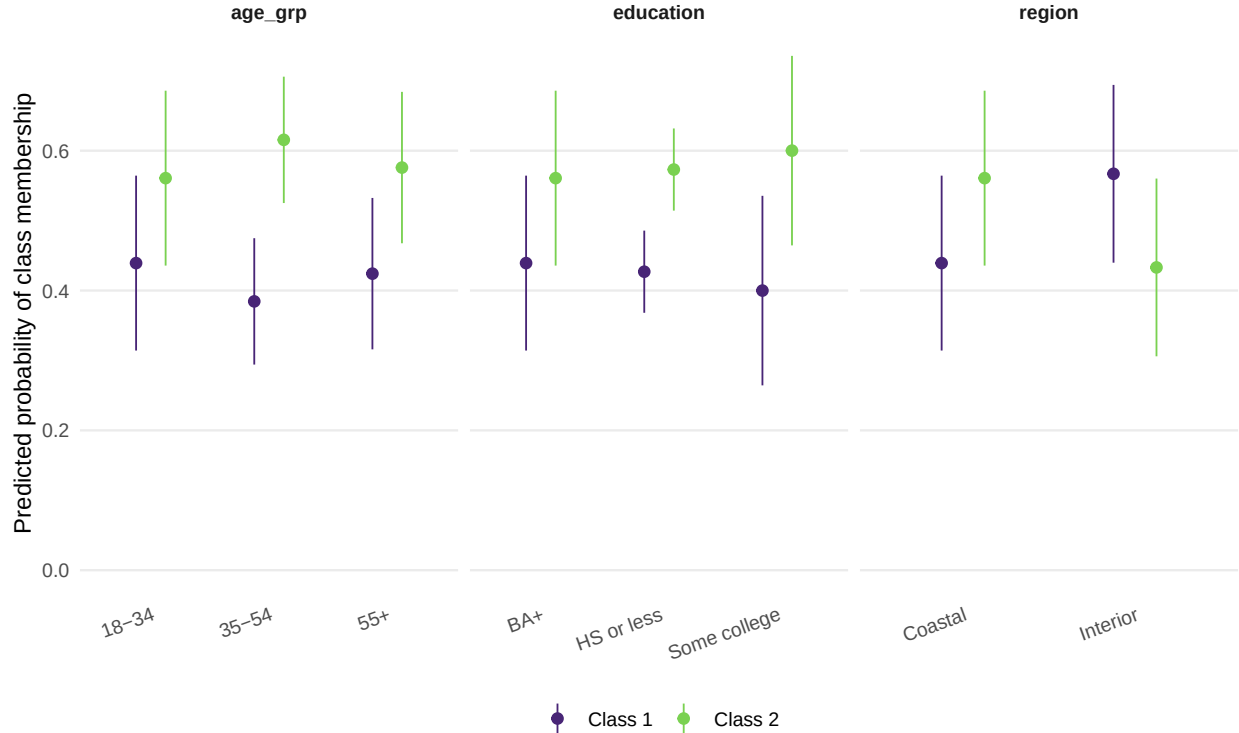


Figure 6: Design-based predicted probabilities of class membership by demographic level, each panel a predictor, with 95% intervals. Within a level the probabilities sum to one across classes. Other predictors are held at their reference level.

15.4 A bias-corrected three-step profile (BCH)

The profile above is honest but conservative. Its outcome is the *modal* class, and because the modal class is an estimate of the true class rather than the true class itself, regressing it on demographics pulls every coefficient toward zero. How much it attenuates depends on how cleanly the classes separate. When inference about the class-demographic association is the goal, the **Bolck-Croon-Hagenaars (BCH) correction** removes this bias while keeping the design-based variance machinery already in place (Bolck, Croon & Hagenaars 2004, *Political Analysis* 12(1):3–27; Vermunt 2010, *Political Analysis* 18(4):450–469).

The correction has three pieces. First, summarise the classification error in a $K \times K$ matrix D with $D_{ts} = \Pr(\text{assigned} = s \mid \text{true} = t)$, estimated from the posteriors and the modal assignments. A near-diagonal D means clean classification; off-diagonal mass is the error that causes attenuation. Second, turn each respondent's modal class into a vector of **soft labels** given by the matching row of D^{-1} . These rows sum to one but can contain negative entries, which is exactly the correction undoing the misclassification. Third, fit the multinomial of class on demographics using those soft labels as the response, weighted by the survey weights.

A natural worry is that negative response weights make the fit unstable. They do not. The curvature of the weighted multinomial log-likelihood depends only on the positive survey weights and the fitted probabilities, not on the signed soft labels, so the objective stays concave with a

single maximum and a standard quasi-Newton solver finds it. That is why the BCH coefficients are well defined even though the implied expanded-data weights are partly negative.

Two assumptions deserve to be stated plainly. BCH assumes **non-differential measurement**: the classification error does not depend on the demographics once the true class is fixed. And it needs the classes **separated enough** that D is invertible; with very poorly separated classes (low entropy) D^{-1} is unstable and BCH, like every three-step correction, becomes unreliable (Vermunt 2010; Bakk, Tekle & Vermunt 2013, *Sociological Methodology* 43(1):272–311). The code checks the full-sample D and stops with a clear message if a class is never modal or D is near-singular, so a degenerate solution is refused rather than reported.

Design-based uncertainty rides on the **same JK_n replicate weights** used everywhere else. On each replicate the classification matrix is recomputed with the replicate weights and the corrected multinomial is refit, holding the posteriors and modal assignments fixed; `survey::withReplicates` turns the spread across replicates into design-based standard errors. One limitation is explicit: this propagates the design variance of the *third step* but treats the latent class fit as given, so it does not carry the step-one uncertainty in the measurement model itself. Propagating that as well would mean refitting the whole model inside a design bootstrap, a heavier exercise not done here.

Because the BCH machinery is reusable across studies, it lives in `survey_lca_source.R` alongside the rest of the engine, already sourced in the setup chunk; its functions are dataset-agnostic and take all inputs as arguments.

```
post_cols <- paste0("post_class", seq_len(K_star))

# BCH needs the K x K classification matrix to be invertible, which requires
# every class to be some respondent's modal class and the classes to be
# separated enough. Check on the full sample first and stop with a readable
# message rather than failing cryptically inside the replicates.
D_full <- classification_matrix(as.matrix(dat_prepared[, post_cols]),
                               as.integer(as.character(dat_prepared$
    ↪ modal_class))),
                               weights(prof_des))
if (any(!is.finite(D_full)) || any(colSums(D_full) < 1e-10))
  stop("At least one class is empty or never the modal class, so the ",
       "classification matrix is singular and BCH cannot run. This signals a ",
       "degenerate class; revisit the number of classes.")
if (rcond(D_full) < 1e-8)
  stop("The classification matrix is near-singular (very poorly separated ",
       "classes). BCH is unreliable here; report the modal profile and treat ",
       "the classes as weakly identified.")

# Statistic for withReplicates: with the replicate weights, recompute D, the
# soft labels, and the corrected multinomial, holding the posteriors and modal
# assignments fixed (the measurement model is treated as given at step three).
# To inspect the layout, call it on the full sample directly:
# bch_theta(weights(prof_des), dat_prepared).
bch_theta <- function(w_rep, data)
  bch_flatten(bch_fit(data, cfg$aux, post_cols, "modal_class",
```

```

w = as.numeric(w_rep), ref = cfg$ref_class))

bch_rep <- withReplicates(prof_rep, bch_theta)           # design-based
  ↪ variance of the BCH coefs
bch_est <- bch_theta(weights(prof_des), dat_prepared)    # full-sample point
  ↪ estimates
bch_se <- sqrt(diag(vcov(bch_rep)))                     # design SEs, same
  ↪ order
stopifnot(length(bch_est) == length(bch_se))

bch_tbl <- tibble(coef = names(bch_est),
                  estimate = as.numeric(bch_est),
                  se = as.numeric(bch_se)) |>
  separate(coef, into = c("class", "term"), sep = "::") |>
  filter(term != "(Intercept)") |>
  mutate(comparison = paste0("Class ", class, " vs ", cfg$ref_class),
         OR = exp(estimate),
         lo = exp(estimate - 1.96 * se),
         hi = exp(estimate + 1.96 * se))

mn_tbl |>
  transmute(class, term, OR_modal = OR) |>
  inner_join(dplyr::select(bch_tbl, class, term, OR_bch = OR, lo, hi),
            by = c("class", "term")) |>
  transmute(Comparison = paste0("Class ", class, " vs ", cfg$ref_class),
            Predictor = term,
            `OR (modal)` = round(OR_modal, 2),
            `OR (BCH)` = round(OR_bch, 2),
            `BCH 95% CI` = sprintf("[% .2f, % .2f]", lo, hi)) |>
  knitr::kable(align = "llrrr",
               caption = paste0("Naive modal versus BCH bias-corrected odds
  ↪ ratios, each class against Class ",
                                cfg$ref_class, ". BCH removes the
  ↪ classification-error attenuation, so its odds ratios sit farther from 1; the
  ↪ intervals are design-based from the JKn replicates and are wider, the price
  ↪ of the correction."))

```

Table 10: Naive modal versus BCH bias-corrected odds ratios, each class against Class 1. BCH removes the classification-error attenuation, so its odds ratios sit farther from 1; the intervals are design-based from the JKn replicates and are wider, the price of the correction.

Comparison	Predictor	OR (modal)	OR (BCH)	BCH 95% CI
Class 2 vs 1	age_grp35-54	1.25	1.36	[0.58, 3.15]
Class 2 vs 1	age_grp55+	1.06	1.09	[0.74, 1.59]
Class 2 vs 1	educationHS or less	1.05	1.07	[0.62, 1.85]
Class 2 vs 1	educationSome college	1.18	1.24	[0.81, 1.90]

Table 10: Naive modal versus BCH bias-corrected odds ratios, each class against Class 1. BCH removes the classification-error attenuation, so its odds ratios sit farther from 1; the intervals are design-based from the JK_n replicates and are wider, the price of the correction.

Comparison	Predictor	OR (modal)	OR (BCH)	BCH 95% CI
Class 2 vs 1	regionInterior	0.60	0.50	[0.34, 0.74]

```
bind_rows(dplyr::select(mn_tbl, comparison, term, OR, lo, hi) |> mutate(method =
  ↪ "Modal (naive)"),
  dplyr::select(bch_tbl, comparison, term, OR, lo, hi) |> mutate(method =
  ↪ "BCH (corrected)")) |>
  mutate(method = factor(method, levels = c("Modal (naive)", "BCH (corrected)")))
  ↪ |>
  ggplot(aes(OR, term, color = method)) +
  geom_vline(xintercept = 1, linetype = "dashed", color = "grey55") +
  geom_pointrange(aes(xmin = lo, xmax = hi), size = 0.35, linewidth = 0.4,
    position = position_dodge(width = 0.6)) +
  facet_wrap(~ comparison) +
  scale_x_log10() +
  scale_color_viridis_d(begin = 0.15, end = 0.7) +
  labs(x = "Odds ratio versus reference class (log scale)", y = NULL, color =
    ↪ NULL) +
  theme_lca()
```

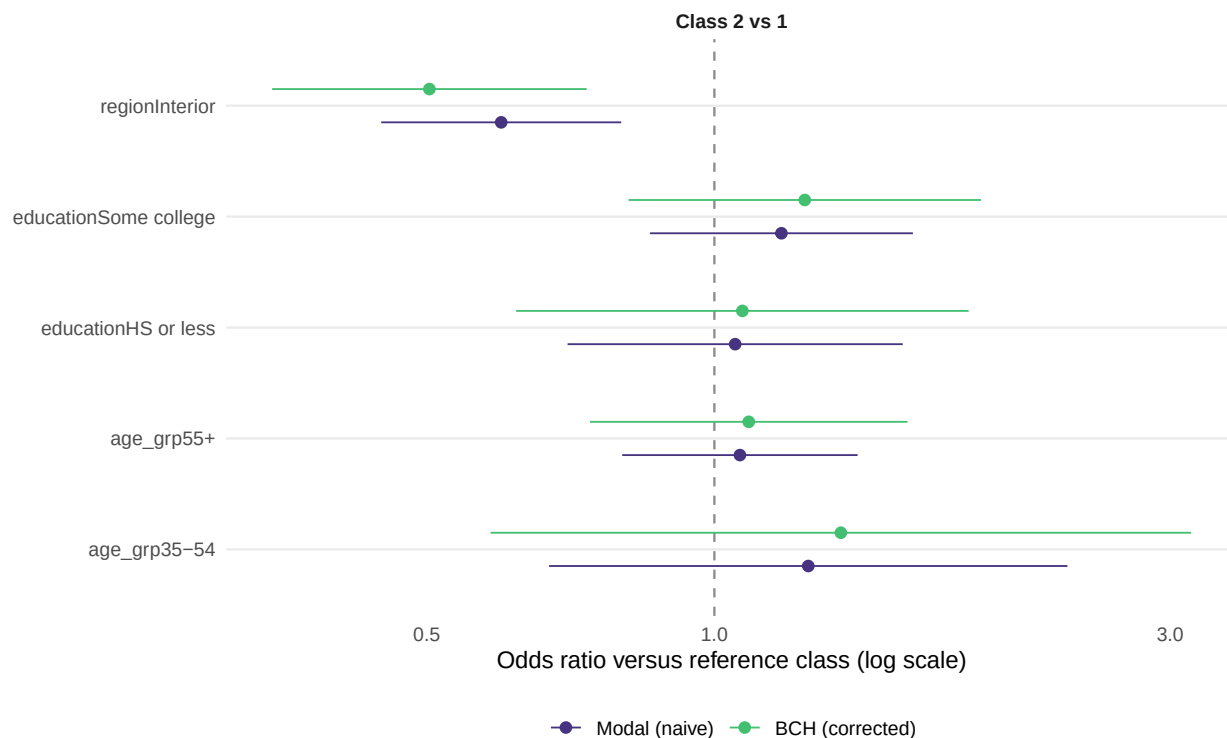


Figure 7: Naive modal (attenuated) versus BCH bias-corrected odds ratios with design-based 95% intervals, each non-reference class against the reference class, by predictor. The correction generally pushes odds ratios away from 1 and widens the intervals.

16 Exporting results

Why this section matters: it makes the analysis usable beyond the render. The enumeration, the design-based interval table, and the fully scored respondent file are the artifacts that feed manuscripts, slides, and downstream merges, and writing them out is what turns a one-time report into a reproducible dataset others can build on.

```
readr::write_csv(enum, "lca_enumeration.csv")
readr::write_csv(ci_tbl |> mutate(across(where(is.numeric), ~ round(.x, 5))),
  "lca_measurement_model_with_CIs.csv")
readr::write_csv(dat_scored_all |>
  dplyr::select(any_of("id"), all_of(c(cfg$strata, cfg$psu,
    ↪ cfg$weight))),
  starts_with("post_class"), modal_class,
  ↪ max_posterior),
  "lca_caselevel_assignments.csv")
```

Three files are written: the enumeration table, the full measurement model with design-based confidence intervals (one row per probability, with `item`, `category`, and `class` as columns), and

the case-level assignments for **all** respondents, including those scored outside the fit in Section 14 (posteriors, modal class, and maximum posterior), ready to merge back into the survey data.

17 Adapting this script to your survey: a checklist

1. **Point `cfg$data` at your data frame and set `cfg$simulated <- FALSE`.** That one switch skips the worked-example simulation (Section 5) and the truth comparison (Section 13), which exist only because the example is simulated. The out-of-sample scoring (Section 14) works on real data and stays.
2. **Set `cfg$items` to your indicator columns, each coded as consecutive integers from 1 over its substantive answers.** Items may have **different numbers of categories**. Put any “Don’t know” or missing code in `cfg$na_codes` (or use NA directly); those responses are treated as missing, the model is fit on complete cases, and missing-item respondents are assigned a class in Section 14. The category-count and missing table after Section 6 lets you confirm the coding.
3. **Set `cfg$strata`, `cfg$psu`, `cfg$weight` to your design columns.** This is the analogue of `svyset psu [pweight = w], strata(stratum)`. If your file ships **replicate weights** instead of strata and PSU identifiers, replace each `svydesign(...)` plus `as.svrepdesign(...)` pair with a single `svrepdesign(repweights = ..., weights = ..., type = ...)`; the rest of the variance code is unchanged.
4. **Set `cfg$aux` to the demographics you want in the profile.** Demographics used to build the weights (age, for instance) are fine and often helpful here; the only variables to exclude are the survey items that defined the classes.
5. **Set `cfg$ref_class` to the class you want as the multinomial baseline.** Class labels are arbitrary, since no substantive order is imposed, so this only chooses which class the odds ratios and predicted probabilities are measured against.
6. **Run, then read enumeration first.** Choose K on the weighted BIC, sanity-checked against entropy and interpretability; the validation check should show the EM matching **poLCA** at unit weights before you trust the weighted output.
7. **Mind the degrees of freedom.** Your design-based intervals carry roughly (number of PSUs minus number of strata) degrees of freedom; with few PSUs they will be wide, and that width is honest.
8. **Treat the modal profile as descriptive, and use the BCH section for inference.** The naive multinomial in this section is attenuated by classification error. For inferential statements about demographic predictors, read the BCH bias-corrected odds ratios at the end of Section 15, which de-attenuate the associations while keeping the design-based variance. BCH assumes the classes are separated enough to be reliably told apart (high entropy); the code stops if they are not.
9. **Keep `survey_lca_source.R` next to this document.** It holds the plotting and data-prep helpers, the weighted EM engine, the BCH correction, and the worked-example simulator, and is sourced in the setup chunk. The functions are dataset-agnostic, so the same file serves any study, but the render fails if it is not in the working directory.
10. **Score respondents outside the fit with `score_lca(newdata, fit_w, cfg$items)`** (Section 14): it assigns a class to anyone carrying the item columns, including a fresh data frame, as long as those items are recoded the same way.

sessionInfo()

R version 4.6.0 (2026-04-24 ucrt)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 26200)

Matrix products: default
LAPACK version 3.12.1

locale:

[1] LC_COLLATE=English_United States.utf8
[2] LC_CTYPE=English_United States.utf8
[3] LC_MONETARY=English_United States.utf8
[4] LC_NUMERIC=C
[5] LC_TIME=English_United States.utf8

time zone: America/New_York
tzcode source: internal

attached base packages:

[1] grid stats graphics grDevices utils datasets methods
[8] base

other attached packages:

[1] nnet_7.3-20	clue_0.3-68	poLCA_1.6.0.2
[4] MASS_7.3-65	scatterplot3d_0.3-45	survey_4.5
[7] survival_3.8-6	Matrix_1.7-5	matrixStats_1.5.0
[10] lubridate_1.9.5	forcats_1.0.1	stringr_1.6.0
[13] dplyr_1.2.1	purrr_1.2.2	readr_2.2.0
[16] tidyr_1.3.2	tibble_3.3.1	ggplot2_4.0.3
[19] tidyverse_2.0.0	gander_0.2.0	ellmer_0.4.1

loaded via a namespace (and not attached):

[1] gtable_0.3.6	xfun_0.57	httr2_1.2.2	lattice_0.22-9
[5] tzdb_0.5.0	vctrs_0.7.3	tools_4.6.0	generics_0.1.4
[9] parallel_4.6.0	cluster_2.1.8.2	pkgconfig_2.0.3	RColorBrewer_1.1-3
[13] S7_0.2.2	lifecycle_1.0.5	compiler_4.6.0	farver_2.1.2
[17] tinytex_0.59	mitools_2.4	htmltools_0.5.9	yaml_2.3.12
[21] crayon_1.5.3	pillar_1.11.1	tidyselect_1.2.1	digest_0.6.39
[25] stringi_1.8.7	labeling_0.4.3	splines_4.6.0	fastmap_1.2.0
[29] cli_3.6.6	magrittr_2.0.5	withr_3.0.2	scales_1.4.0
[33] rappdirs_0.3.4	bit64_4.8.2	timechange_0.4.0	rmarkdown_2.31
[37] bit_4.6.0	otel_0.2.0	hms_1.1.4	evaluate_1.0.5
[41] knitr_1.51	viridisLite_0.4.3	rlang_1.2.0	Rcpp_1.1.1-1.1
[45] glue_1.8.1	DBI_1.3.0	coro_1.1.0	rstudioapi_0.18.0
[49] vroom_1.7.1	jsonlite_2.0.0	R6_2.6.1	